PROGRAMMING IN JAVA

ASSIGNMENT 4

EVENT-DRIVEN GUI DESIGN

INTRODUCTION

Graphical User Interface (GUI) development is an important part of Java programming. It allows users to interact with software through windows, buttons, text fields, menus and other graphical components instead of entering commands through a terminal.

Java provides two major libraries for GUI development: Abstract Window Toolkit (AWT) and Swing. AWT provides basic graphical components, while Swing offers more flexible and advanced components for developing desktop applications.

Event-driven programming is a programming approach in which the execution of an application is controlled by events generated through user interactions. These events may include mouse clicks, keyboard inputs, button selections and window operations.

Java uses the Event Delegation Model to handle these events. In this model, an event source generates an event and sends it to a registered listener. The listener performs the required operation.

This assignment focuses on designing effective graphical interfaces, implementing event handlers, selecting appropriate layout managers, identifying event-handling errors and developing a layered GUI application connected to a database.

OBJECTIVES

1. To understand the Event Delegation Model in Java.
2. To identify the responsibilities of event sources and event listeners.
3. To design graphical interfaces using AWT and Swing components.
4. To understand the use of adapter classes, inner classes and lambda expressions.
5. To select suitable layout managers for different GUI requirements.
6. To identify and correct errors in event-handling mechanisms.
7. To implement proper input validation and user feedback.
8. To understand database connectivity using JDBC.
9. To design responsive applications using background processing.
10. To compare alternative GUI designs based on performance, memory usage and maintainability.

TASK 1 – GUI DESIGN REVIEW

Diagnose and Repair a Poor Event-Handling Design

1.1 Analysis of the Existing Fee Payment Application

The given fee-payment application is designed to collect a student's registration number and payment amount. It provides three buttons: Pay, Clear and Receipt.

However, the existing design places all operations inside a single event-handling method. This creates several problems related to code organization, security, performance and user experience.

Identified Design Problems

S.No	Problem	Detailed Explanation
1	Lack of separation of responsibilities	The GUI and payment-processing operations are combined in one class. This makes the program difficult to understand and modify.
2	Incorrect String comparison	The action command is compared using ==. Java String contents should be compared using equals() to avoid incorrect comparisons.
3	Missing input validation	The application does not properly check whether the registration number and payment amount are valid. Empty or incorrect input may cause errors.
4	Database operations inside the event handler	Connecting to a database and inserting records directly inside the event handler may delay the interface response.
5	Blocking operation using sleep	Calling Thread.sleep() inside the event-dispatch thread pauses GUI processing. The application may become temporarily unresponsive.
6	Insufficient confirmation messages	The application prints the payment result in the console instead of displaying a clear confirmation message to the user.
7	Receipt functionality is missing	The Receipt button does not perform the required operation. Users cannot view their payment information.
8	Unsafe SQL query construction	Directly combining user input with an SQL statement creates a risk of SQL injection attacks.
9	Improper database resource management	The database connection is not properly closed after completing the transaction.

10	Missing exception handling	Database and input-related exceptions are not handled properly, which can cause unexpected application termination.
11	Missing window event handling	The application does not define proper behaviour when the user closes the payment window.

1.2 Proposed Improved Design

The application can be improved by separating the graphical interface from the payment-processing logic.

The GUI should collect the student's details and display the payment status. A separate service class should validate the information and perform the payment operation.

Proposed Class Structure

FeeWindow

- Displays the payment form.
- Accepts the registration number.
- Accepts the payment amount.
- Handles Pay, Clear and Receipt button actions.
- Displays messages to the user.

FeeService

- Validates registration details.
- Checks whether the payment amount is valid.
- Performs payment processing.
- Returns the transaction result.

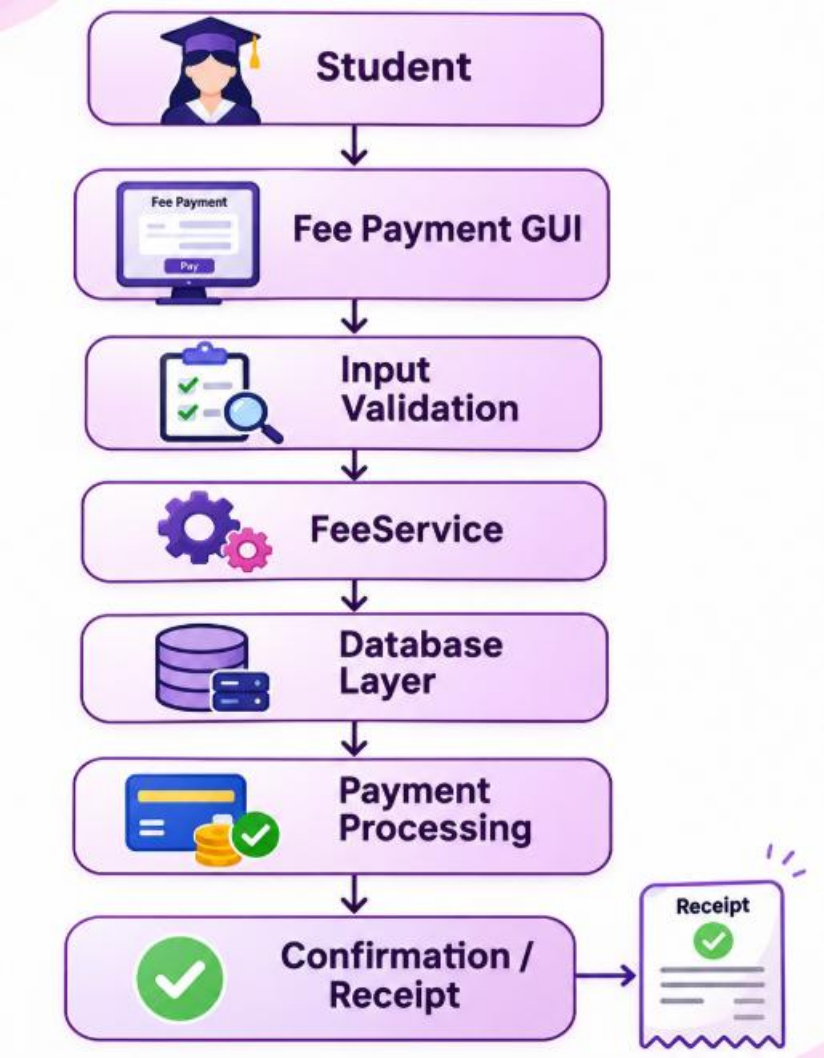
Database Layer

- Establishes database connectivity.
- Stores payment details.
- Uses parameterized SQL queries.
- Closes database resources properly.

Receipt Module

- Displays payment information.
- Shows the registration number.
- Shows the amount paid.
- Displays the transaction status.

Improved Design Flow



Event-Handling Pseudocode

Algorithm: Process Fee Payment

```
BEGIN

    RECEIVE registration number
    RECEIVE payment amount

    IF registration number is empty THEN
        DISPLAY registration number error
        STOP
    END IF

    IF payment amount is invalid OR amount <= 0 THEN
        DISPLAY invalid payment amount
        STOP
    END IF
```

```
   CALL FeeService to process payment

   IF payment is successful THEN
      DISPLAY payment confirmation
      STORE transaction details
   ELSE
      DISPLAY payment failure message
   END IF

END
```

Algorithm: Clear Payment Form

```
BEGIN

   CLEAR registration number field
   CLEAR payment amount field
   CLEAR status message

END
```

Algorithm: Generate Payment Receipt

```
BEGIN

   CHECK payment transaction status

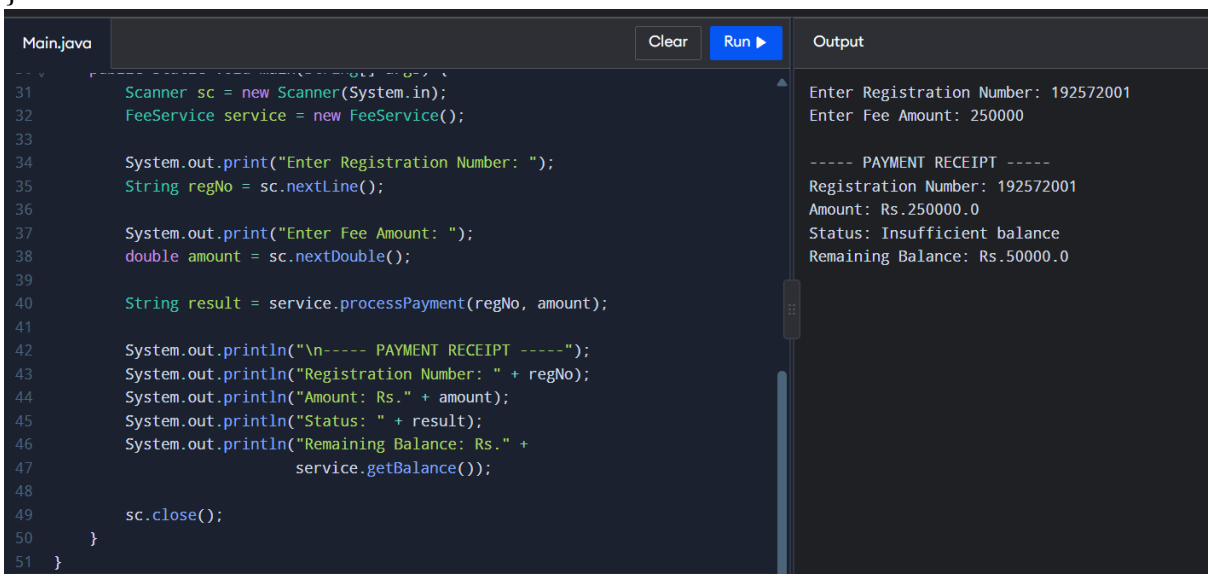
   IF payment is successful THEN
      DISPLAY registration number
      DISPLAY payment amount
      DISPLAY transaction details
      DISPLAY payment status
   ELSE
      DISPLAY no successful payment available
   END IF

END
```

PROGRAM:

```
import java.util.Scanner;
class FeeService {
    private double balance = 50000;
    public String processPayment(String regNo, double amount) {
        if (regNo == null || regNo.trim().isEmpty()) {
            return "Invalid registration number";
        }
        if (amount <= 0) {
            return "Enter a valid payment amount";
        }
if (amount > balance) {
```

```
            return "Insufficient balance";
        }
        balance -= amount;
        return "Payment successful";
    }
    public double getBalance() {
        return balance;
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        FeeService service = new FeeService();
        System.out.print("Enter Registration Number: ");
        String regNo = sc.nextLine();
        System.out.print("Enter Fee Amount: ");
        double amount = sc.nextDouble();
        String result = service.processPayment(regNo, amount);
        System.out.println("\n----- PAYMENT RECEIPT -----");
        System.out.println("Registration Number: " + regNo);
        System.out.println("Amount: Rs." + amount);
        System.out.println("Status: " + result);
        System.out.println("Remaining Balance: Rs." +
                           service.getBalance());
        sc.close();
    }
}
```



1.3 Advantages of the Improved Design

The redesigned application follows separation of concerns. Each class performs a specific responsibility, making the system easier to understand and maintain.

The payment service can be tested independently without opening the graphical interface. Input validation improves reliability, while parameterized SQL statements improve database security. Background processing can prevent lengthy operations from freezing the window.

Alternative Event-Handling Approach

Instead of using one listener with multiple conditional statements, each button can be assigned an independent listener object or a Swing Action.

Single listener approach:

- One event handler receives events from multiple buttons.
- It checks the action command and identifies the operation.
- For k buttons, the worst-case number of comparisons is proportional to k.

Separate listener approach:

- Each button has its own registered event handler.
- The required operation can be called directly.
- Event dispatch is approximately constant time, excluding the actual operation performed.

Complexity Comparison

Parameter	Single Listener	Separate Listeners
Event identification	Up to k comparisons	Direct handler selection
Time complexity	O(k) worst case	Approximately O(1)
Additional space	Usually fewer listener objects	O(k) listener objects
Readability	Can become complicated	Easier to organize
Maintenance	Changes may affect shared handler	Individual actions can be modified separately

Conclusion: Separate listeners are useful when an application contains multiple independent actions because they improve code organization and maintainability.

TASK 2 – FROM REQUIREMENT TO EVENT-DRIVEN DESIGN

Hospital Outpatient Appointment Desk

2.1 Introduction

A hospital outpatient appointment desk requires a graphical application to manage patient appointments. The receptionist should be able to enter a patient ID, select a medical department, choose a doctor and select an available appointment time.

The application should display appropriate confirmation and error messages. It should also prevent the window from closing without confirmation when an appointment booking is in progress.

2.2 Event Model

The application uses different event sources and listeners to respond to user activities.

S.No	Event Source	Event Object	Listener Interface	Action
1	Patient ID text field	KeyEvent	KeyListener	Handles keyboard input.
2	Department selection	ItemEvent	ItemListener	Updates the doctor list.
3	Doctor selection	ItemEvent	ItemListener	Records the selected doctor.
4	Time-slot selection	ItemEvent	ItemListener	Records the selected appointment time.
5	Book Appointment button	ActionEvent	ActionListener	Validates and submits the booking.
6	Window close operation	WindowEvent	WindowListener	Checks whether booking is in progress.
7	Confirmation dialog	ActionEvent	ActionListener	Handles confirmation or cancellation.

2.3 GUI Component Hierarchy

The hospital appointment application can be designed using a main JFrame and multiple JPanel containers.

JFrame – Hospital Appointment Desk
|
|-- JPanel – Patient Information
| |-- JLabel – Patient ID
| |-- JTextField – Patient ID
|
|-- JPanel – Department Information
| |-- JLabel – Department
| |-- JComboBox – Department List
|
|-- JPanel – Doctor Information
| |-- JLabel – Doctor Name
| |-- JComboBox – Doctor List
|
|-- JPanel – Appointment Information
| |-- JLabel – Time Slot
| |-- JComboBox – Available Slots
|
|-- JPanel – Action Buttons
| |-- JButton – Book Appointment
| |-- JButton – Clear
|
|-- JLabel – Status Information

Components and Their Uses

Component	Application
JFrame	Creates the main appointment window.
JPanel	Groups related GUI components.
JLabel	Displays field names and messages.
JTextField	Accepts the patient ID.
JComboBox	Provides department, doctor and time selections.
JButton	Executes booking and clearing operations.
JOptionPane	Displays confirmation and error dialogs.

2.4 Event Handler Design

Algorithm: Update Doctor List

BEGIN

GET selected department

REMOVE existing doctor entries

SEARCH the doctor collection using department name

```
    IF doctors are available THEN
        ADD the doctors to the doctor list
    ELSE
        DISPLAY no doctors available
    END IF

END
```

Algorithm: Book Hospital Appointment

```
BEGIN

    READ patient ID
    READ selected department
    READ selected doctor
    READ appointment time

    IF patient ID is empty THEN
        DISPLAY patient ID error
        STOP
    END IF

    IF department is not selected THEN
        DISPLAY department selection error
        STOP
    END IF

    IF doctor is not selected THEN
        DISPLAY doctor selection error
        STOP
    END IF

    IF appointment time is not selected THEN
        DISPLAY time selection error
        STOP
    END IF

    SET booking status to IN PROGRESS

    CHECK doctor availability

    IF doctor is available THEN
        SAVE appointment information
        DISPLAY booking confirmation
    ELSE
        DISPLAY selected slot is unavailable
    END IF

    SET booking status to COMPLETED

END
```

2.5 Event Execution Sequence

The event sequence begins when the application is launched and ends when the user closes the window.

1. The main method starts the application.
2. The JFrame object is created.
3. Required Swing components are initialized.
4. Layout managers arrange the components.
5. Event listeners are registered.
6. The application window becomes visible.
7. The receptionist enters the patient ID.
8. The department selection event updates the doctor list.
9. The receptionist selects a doctor and appointment time.
10. The Book Appointment button generates an ActionEvent.
11. The event handler validates the input.
12. The appointment is processed.
13. A success or error message is displayed.
14. When the close operation is requested, a WindowEvent is generated.
15. The application checks whether a booking is still in progress.
16. If necessary, a confirmation dialog appears.
17. The application closes after confirmation.

2.6 Comparison with Applet Life-Cycle Methods

Applet Method	Function
init()	Initializes applet resources.
start()	Starts or resumes applet execution.
paint()	Displays graphical output.
stop()	Pauses applet execution.
destroy()	Releases applet resources.

A desktop Swing application follows a different execution model. It starts using the main() method and handles user actions through registered listeners and window events.

Alternative Approach: Database Fetching vs Map Storage

The doctor details can be managed in two ways.

Approach 1: Store doctor details in a Map

A Map<String, List<Doctor>> can store department names and their corresponding doctors.

- Doctor information is loaded once.
- Department selection retrieves the corresponding doctor list.
- It avoids repeated database queries.
- The lookup is approximately O(1) on average, followed by the time required to display the selected doctors.

Approach 2: Retrieve doctors from the database

Every department change triggers a database query.

- The latest doctor information can be retrieved.
- Repeated queries may increase network and database activity.
- Response time depends on the database and query execution.

Complexity Comparison

Feature	Map Storage	Database Fetching
Department lookup	O(1) average	Depends on database query
Doctor display	O(x) for x doctors	Depends on result retrieval and display
Memory	O(d + m)	Lower local cache requirement
Database activity	Mostly during initial loading	Every department change
Data freshness	Depends on refresh strategy	Retrieves current database data

Conclusion: Map storage is suitable when doctor information changes infrequently. Database retrieval is appropriate when updated information is required for every selection.

TASK 3 – FROM SKETCH TO LAYOUT

Campus Library Search Screen

3.1 Introduction

The campus library search screen allows users to search for books using a search field and category selection. It also displays search results and provides navigation and reservation options.

The interface must adjust properly when the user changes the window size. The results section should expand, while the search controls and bottom buttons should remain organized.

3.2 Selection of Layout Managers

Java provides different layout managers for arranging graphical components. The appropriate layout should be selected according to the requirements of each section.

Layout Manager	**Usage**	**Reason**
BorderLayout	Main application window	Divides the window into five regions.
FlowLayout	Search controls	Arranges search components horizontally.
BorderLayout	Results panel	Allows the centre area to expand.
FlowLayout	Bottom button panel	Arranges navigation buttons in one row.
GridLayout	Structured form sections	Places components in equal-sized rows and columns.
CardLayout	Multiple screens	Displays one panel at a time.

3.3 Nested Container Structure

```
JFrame – Library Search Screen
|
|-- NORTH
|   |-- JLabel – Library Title
|
|-- CENTER
|   |
|   |-- JPanel – Main Content
|       |
|       |-- NORTH: Search Panel
|       |   |-- JTextField – Search Text
|       |   |-- JComboBox – Category
|       |   |-- JButton – Search
|       |
|       |-- CENTER: Results Panel
|           |-- JTable / JTextArea – Results
|
|-- SOUTH
    |
    |-- JPanel – Navigation Panel
    |   |-- JButton – Prev
    |   |-- JButton – Next
    |   |-- JButton – Reserve
    |
```

|-- JLabel – Status Message

3.4 Layout Behaviour During Resizing

BorderLayout is useful because it distributes extra space to the centre component. Therefore, the library results section can expand when the main window becomes larger.

FlowLayout keeps the search field, category selection and Search button arranged together. The bottom button panel also uses FlowLayout to keep the navigation controls in a single row.

3.5 Resizing Test Plan

The application should be evaluated using three different window sizes.

Test	Window Dimensions	Expected Result
Test 1	600 × 400	Results area remains visible with limited space.
Test 2	900 × 600	Results area expands and provides more display space.
Test 3	1200 × 800	Results area grows further while controls remain arranged in their panels.

Observation: The selected layout managers provide an organized interface and allow the centre results area to use the available extra space.

Alternative Layout Approaches

GridBagLayout

GridBagLayout provides flexible positioning of components. It allows different component sizes and supports resizing through constraints. However, it requires more configuration.

Absolute Positioning

Absolute positioning uses fixed coordinates and dimensions for every component. Although it is simple for small demonstrations, it does not automatically adjust components when the window is resized.

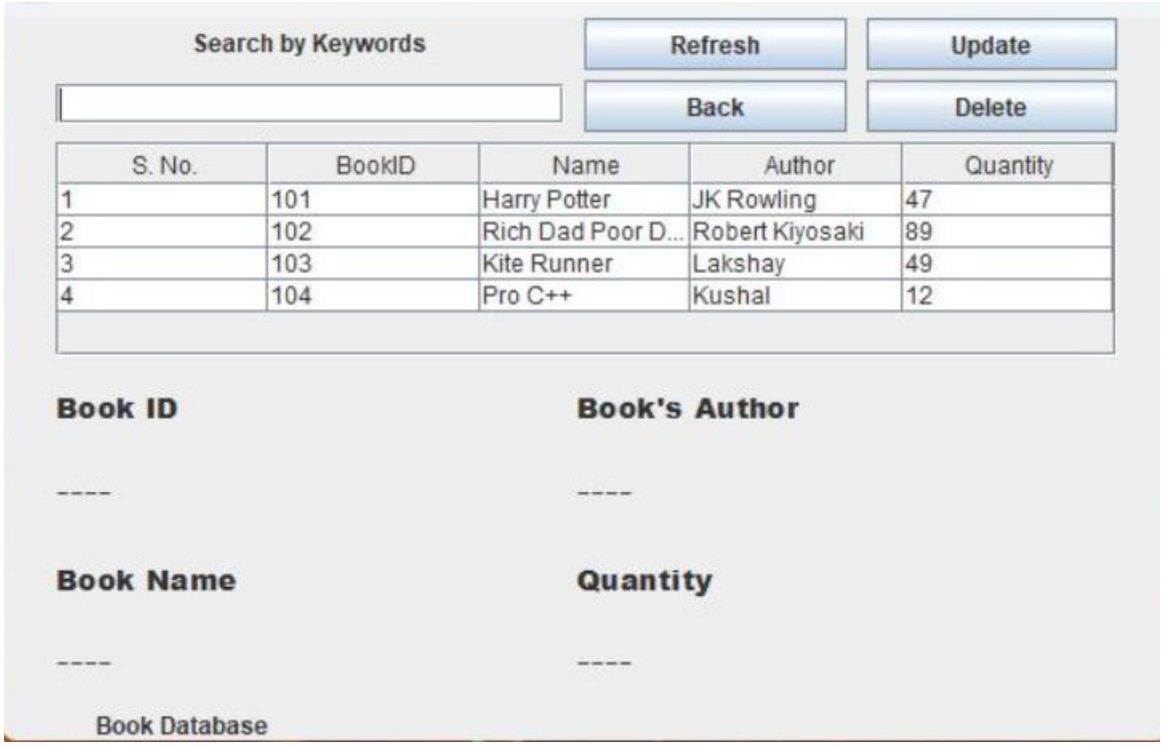
Comparison

Parameter	BorderLayout + FlowLayout	GridBagLayout	Absolute Positioning
Resizing	Good	Good with suitable constraints	Poor without manual changes
Flexibility	Moderate	High	Low
Coding effort	Low	Higher	Low initially
Maintenance	Easy	Moderate	Difficult
Layout calculations	Relatively simple	Constraint-based	Simple initial positioning
Suitability	Standard responsive screens	Complex forms	Fixed-size demonstrations

PROGRAM:

```
import javax.swing.*;
import java.awt.*;
public class Main extends JFrame {
   JTextField searchField;
   JComboBox<String> category;
   JTextArea results;
   JLabel status;
   Main() {
      setTitle("Campus Library Search");
      setSize(700, 500);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      setLayout(new BorderLayout(10, 10));
      JLabel title = new JLabel("CAMPUS LIBRARY SEARCH",
                        JLabel.CENTER);
      title.setFont(new Font("Arial", Font.BOLD, 20));
      add(title, BorderLayout.NORTH);
      JPanel centerPanel = new JPanel(new BorderLayout(5, 5));
      JPanel searchPanel = new JPanel(new FlowLayout());
      searchField = new JTextField(20);
      category = new JComboBox<>(
         new String[]{"All", "Java", "Python", "Database", "AI"}
      );
      JButton searchButton = new JButton("Search");
```

```
    searchPanel.add(new JLabel("Search Book:"));
    searchPanel.add(searchField);
    searchPanel.add(category);
    searchPanel.add(searchButton);
    centerPanel.add(searchPanel, BorderLayout.NORTH);
    results = new JTextArea();
    results.setEditable(false);
    results.setText("Search results will appear here.");
    JScrollPane scrollPane = new JScrollPane(results);
    centerPanel.add(scrollPane, BorderLayout.CENTER);
    add(centerPanel, BorderLayout.CENTER);
    JPanel bottomPanel = new JPanel(new FlowLayout());
    JButton prev = new JButton("Prev");
    JButton next = new JButton("Next");
    JButton reserve = new JButton("Reserve");
    bottomPanel.add(prev);
    bottomPanel.add(next);
    bottomPanel.add(reserve);
    JPanel southPanel = new JPanel(new BorderLayout());
    status = new JLabel("Ready");
    southPanel.add(bottomPanel, BorderLayout.CENTER);
    southPanel.add(status, BorderLayout.SOUTH);
    add(southPanel, BorderLayout.SOUTH);
    searchButton.addActionListener(e -> {
        String book = searchField.getText();
        String type = (String) category.getSelectedItem();
        if (book.trim().isEmpty()) {
            status.setText("Enter a book name");
            return;
        }
        results.setText("Book Name: " + book +
                        "\nCategory: " + type +
                        "\nSearch completed.");
        status.setText("Search successful");
    });
    reserve.addActionListener(e ->
        status.setText("Book reservation requested")
    );
    prev.addActionListener(e ->
        status.setText("Previous page selected")
    );
    next.addActionListener(e ->
        status.setText("Next page selected")
    );
}
public static void main(String[] args) {
    SwingUtilities.invokeLater(() ->
        new Main().setVisible(true)
    );
}}
```



Conclusion: A combination of BorderLayout and FlowLayout is appropriate for the campus library screen because it provides simple component organization and the required resizing behaviour.

TASK 4 – EVENT-HANDLING FORENSICS

Drawing Pad Application

4.1 Introduction

The drawing pad application is designed to draw a small circular dot whenever a user clicks inside the window. It should also maintain a label showing the total number of mouse clicks.

The given implementation contains event-handling and drawing-related problems. These problems prevent the application from responding correctly and preserving the drawing.

4.2 Identification of the Event-Handling Defect

The original code contains an incorrectly written mouse event method.

The method is written as mouseclicked(), but the correct method name in Java is mouseClicked().

Java is case-sensitive. Therefore, the incorrectly named method does not override the original method in MouseAdapter. As a result, the expected event handler is not executed.

Another problem is that the click counter is increased without updating the label. The label must be refreshed using the appropriate text-setting operation.

4.3 Why Drawn Dots Disappear

The application uses getGraphics() to draw directly on the window.

This method does not preserve the drawing as permanent application data. When the window is minimized, restored or repainted, the previous drawing may disappear.

The solution is to store the coordinates of every mouse click and redraw all stored points whenever the window is repainted.

4.4 Improved Drawing Design

The repaired application should maintain a collection of clicked points.

Each mouse click should perform the following operations:

1. Read the mouse pointer's x-coordinate.
2. Read the mouse pointer's y-coordinate.
3. Store the coordinates in a point collection.
4. Increase the click counter.
5. Update the status label.
6. Request repainting.

The paint method should iterate through the stored points and draw a filled circle for each point.

Mouse Event Handling Pseudocode

```
BEGIN

  RECEIVE mouse click event

  GET mouse x-coordinate
  GET mouse y-coordinate

  STORE point in collection

  INCREMENT click counter

  UPDATE click status label

  REQUEST repaint
```

```
END
```

Drawing Pseudocode

```
BEGIN
  FOR each stored point
    GET point coordinates
    DRAW filled circle at the point
  END FOR
END
```

4.5 Importance of the Override Annotation

The @Override annotation is used to indicate that a method is intended to override a method from a parent class or interface.

If the method name is written incorrectly, the Java compiler can identify that the method does not override the expected method.

Therefore, using @Override improves program reliability and helps developers identify method-name mistakes during compilation.

Alternative Drawing Technique

The drawing application can also use an off-screen BufferedImage.

Approach 1: Store Points

- Maintain every clicked point in a collection.
- Redraw all points during each repaint.
- The drawing data requires memory proportional to the number of points.

Time per repaint: O(n), where n is the number of stored points.

Space complexity: O(n).

Approach 2: Use BufferedImage

- Create an image with the required width and height.
- Draw new dots onto the image.

- Display the stored image whenever the window is repainted.

Space complexity: O(W × H), where W is the image width and H is its height.

The repaint operation copies or renders the stored image rather than individually redrawing all point objects.

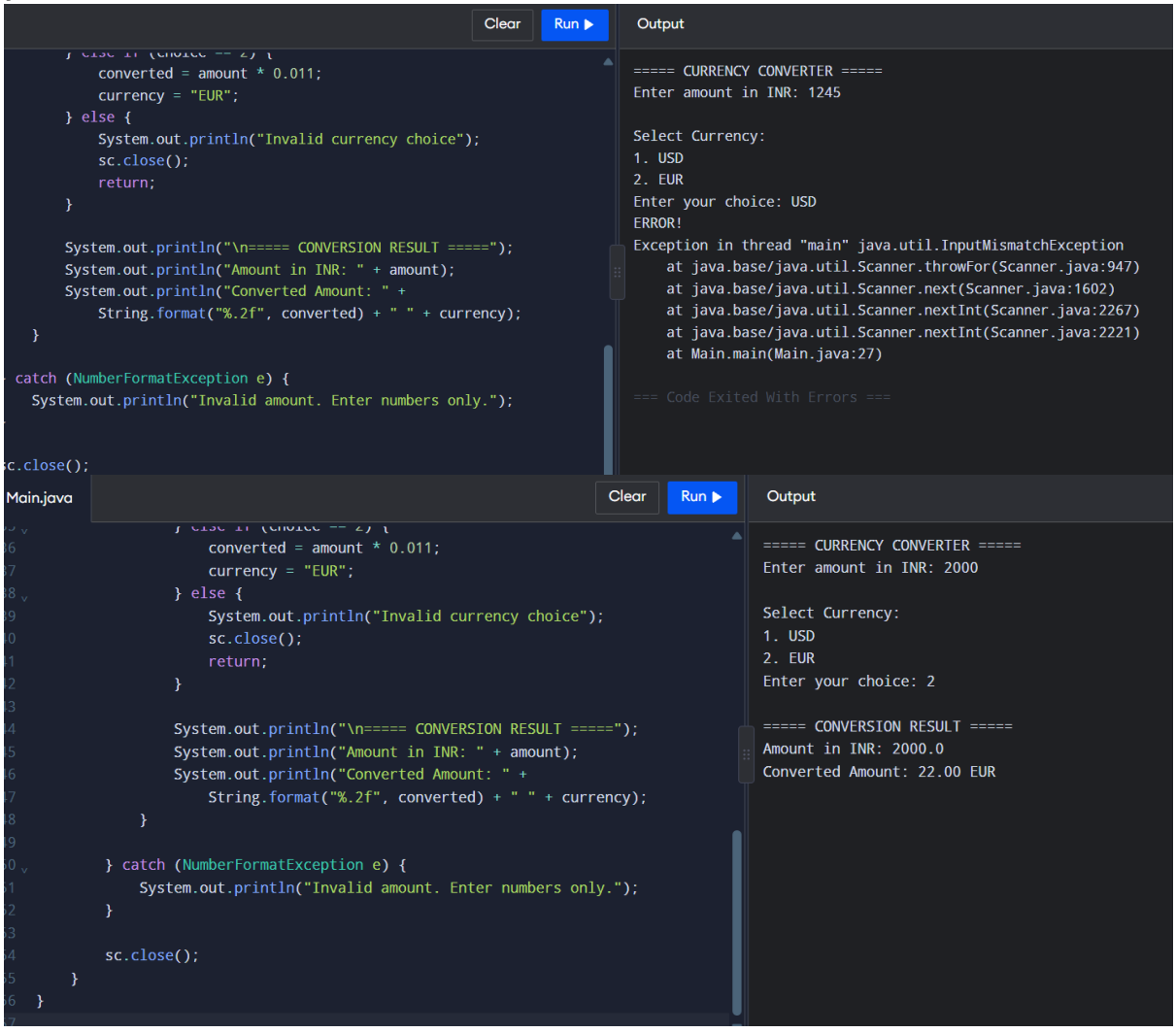
Comparison

Parameter	Point Collection	BufferedImage
Drawing storage	Coordinates	Pixel image
Repainting	Redraws all points	Displays stored image
Time per repaint	O(n)	Image-copy/rendering cost
Space complexity	O(n)	O(W × H)
Implementation	Simple	Requires image management
Best use	Small drawing applications	Persistent drawing surfaces

PROGRAM:

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("===== CURRENCY CONVERTER =====");
        System.out.print("Enter amount in INR: ");
        String input = sc.nextLine();
        try {
            double amount = Double.parseDouble(input);
            if (amount <= 0) {
                System.out.println("Invalid amount");
            } else {
                System.out.println("\nSelect Currency:");
                System.out.println("1. USD");
                System.out.println("2. EUR");
                System.out.print("Enter your choice: ");
                int choice = sc.nextInt();
                double converted;
                String currency;
                if (choice == 1) {
                    converted = amount * 0.012;
                    currency = "USD";
                } else if (choice == 2) {
                    converted = amount * 0.011;
                    currency = "EUR";
                } else {
```

```
            System.out.println("Invalid currency choice");
            sc.close();
            return;
        }
        System.out.println("\n===== CONVERSION RESULT =====");
        System.out.println("Amount in INR: " + amount);
        System.out.println("Converted Amount: " +
            String.format("%.2f", converted) + " " + currency);
      }
    } catch (NumberFormatException e) {
      System.out.println("Invalid amount. Enter numbers only.");
    }
    sc.close();
  }
}
```



Conclusion: Storing points provides a simple solution for a basic drawing pad. BufferedImage is useful when the application requires persistent graphical content and efficient repainting.

TASK 5 – SAME SCREEN, DIFFERENT EVENT-HANDLING STYLES

Currency Converter Application

5.1 Introduction

The currency converter is a graphical application that accepts an amount in Indian Rupees and converts it into a selected foreign currency.

The application contains an amount field, a target currency selection, a Convert button and a result display.

The assignment specifies the following conversion rates:

- USD conversion rate: 0.012
- EUR conversion rate: 0.011

The Convert button should respond to user interaction and display the calculated result.

5.2 Four Event-Handling Techniques

Java provides different approaches for handling GUI events. The currency converter can demonstrate four techniques.

A. Implementing ActionListener in the Frame

In this method, the main JFrame class implements the ActionListener interface.

The class overrides the actionPerformed() method to perform the conversion. The button registers the frame as its listener.

Advantages:

- Simple for small programs.
- The handler can directly access frame variables.
- Easy to understand for beginners.

Limitation:

- A large number of event-handling operations can make the frame class lengthy.

B. Named Inner Class

A named inner class is created inside the main converter class. This inner class implements ActionListener and handles the conversion event.

Advantages:

- Separates event-handling code from the main constructor.
- Can access the outer class variables.
- Suitable for handlers containing multiple operations.

Limitation:

- Requires additional class definition and code.

C. Anonymous Inner Class

An anonymous inner class creates an event listener directly at the location where it is registered.

Advantages:

- No separate named class is required.
- Suitable for short event-handling operations.
- Can access the outer class members.

Limitation:

- Complex operations inside anonymous classes may reduce readability.

D. Lambda Expression

A lambda expression provides a shorter way to implement functional interfaces such as ActionListener.

Advantages:

- Reduces unnecessary code.
- Makes simple event handlers easier to read.
- Useful when the event operation is short.

Limitation:

- Complex operations should be moved into separate methods.

5.3 Comparison of Event-Handling Techniques

Feature	Frame Listener	Named Inner Class	Anonymous Inner Class	Lambda
Code length	Moderate	More	Moderate	Short
Readability	Good for small applications	Good for organized handlers	Good for short handlers	Very good for short actions
Field access	Direct	Direct	Direct	Direct
Reusability	Depends on frame design	Good	Limited	Good when stored and reused
Maintenance	May become crowded	Easier to organize	Can become difficult for complex logic	Good for simple operations

5.4 Handling the Enter Key

The application must also perform conversion when the user presses Enter inside the amount field.

The same conversion handler can be registered with both the Convert button and the amount field.

This avoids duplicating the conversion logic.

5.5 Displaying Results Using Dialog Boxes

The converted value should be displayed using a dialog box.

The application should:

1. Read the amount entered by the user.
2. Convert the input into a numeric value.
3. Check whether the value is valid and positive.
4. Identify the selected currency.
5. Select the appropriate conversion rate.
6. Calculate the converted amount.
7. Format the result to two decimal places.
8. Display the result in a message dialog.

If the entered amount is invalid, the application should display the message "Invalid amount".

Alternative Concept: Method Reference and Shared Action

A method reference can be used to connect a GUI event directly to an existing method.

A shared Swing Action can also be registered with multiple GUI controls, such as a button and menu item.

Advantages:

- Reduces repeated event-handling code.
- Supports reuse of the same operation.
- Makes it easier to add new event triggers.
- Improves maintainability.

Comparison

Parameter	**Independent Listeners**	**Method Reference**	**Shared Action**
Code reuse	Depends on implementation	Good	Very good
Additional triggers	Require registration	Easy	Easy
Code organization	Depends on listener design	Simple	Centralized
Suitable application	Small GUI	Existing methods	Larger GUI with multiple controls

Conclusion: Lambda expressions are suitable for short event operations. Named inner classes and shared Action objects are better choices when the application contains more complex or reusable event-handling logic.

TASK 6 – DESIGN CHALLENGE

Campus Event Registration Desktop Application

6.1 Introduction

The campus event registration application is designed to manage student participation in college events. The cultural committee requires a desktop system that can store registration details, display registrations, filter records and export registration information.

The application should provide a simple graphical interface and connect to a MySQL database.

It should also validate user input, prevent duplicate registrations and remain responsive while retrieving or storing data.

6.2 Identification of Users

The application has three major categories of users.

1. Students

- Enter personal information.
- Select a campus event.
- Submit registration details.
- Receive registration confirmation.

2. Committee Members

- View student registrations.
- Filter records by event.
- Export registration information.
- Review event participation details.

3. Administrator

- Manage application information.
- Maintain event records.
- Monitor registration information.

6.3 Required Application Screens

Screen	Description
Registration Screen	Accepts student and event details.
Registration List Screen	Displays stored registrations.
Event Filter Screen	Filters registrations according to the selected event.
Export Screen	Exports registration records.
Message Dialog	Displays errors and confirmation messages.

6.4 Data Requirements

The application must maintain the following information:

Field	Description

Registration ID	Unique registration number.
Student ID	Student's identification number.
Student Name	Name of the registered student.
Email	Student's email address.
Department	Department of the student.
Event Name	Name of the selected campus event.
Registration Date	Date and time of registration.

6.5 Functional Requirements

1. The application must allow students to enter their details.
2. The system must provide an event selection option.
3. The application must validate mandatory fields.
4. The system must check for duplicate registrations.
5. Successful registration details must be stored in MySQL.
6. Committee members must be able to view registration records.
7. The application must support filtering by event.
8. The system must provide an export option.
9. Invalid input must generate clear error messages.
10. Database operations must not block the GUI.

6.6 Proposed Screen Structure

Registration Window

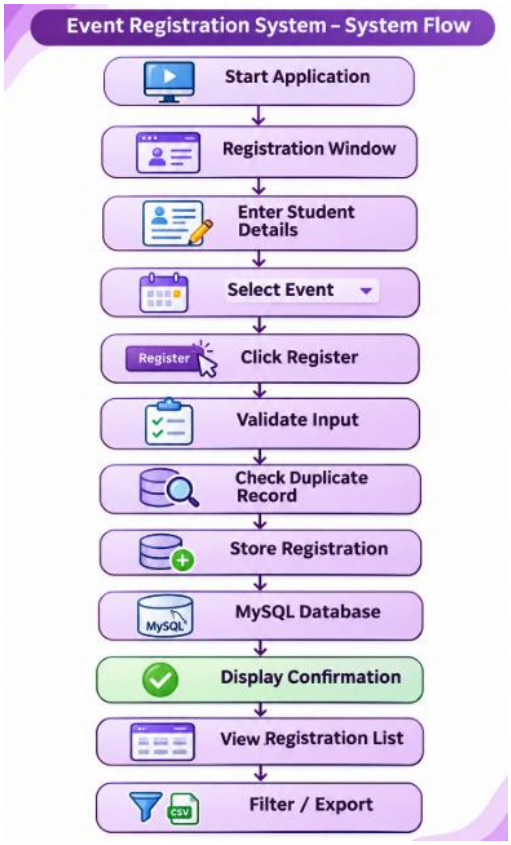
```
JFrame – Campus Event Registration
|
|-- JMenuBar
|   |-- File
|   |   |-- Export
|   |   |-- Exit
|   |
|   |-- Registration
|       |-- New Registration
|       |-- View Registrations
|
|-- JPanel – Student Details
|   |-- JLabel – Student ID
|   |-- JTextField – Student ID
|   |-- JLabel – Student Name
|   |-- JTextField – Student Name
|   |-- JLabel – Email
```

| |-- JTextField – Email
|
|-- JPanel – Event Details
| |-- JLabel – Department
| |-- JComboBox – Department
| |-- JLabel – Event
| |-- JComboBox – Event
|
|-- JButton – Register
|
|-- JLabel – Status

Registration List Window

JFrame – Registration Records
|
|-- JPanel – Filter
| |-- JComboBox – Event Selection
| |-- JButton – Filter
|
|-- JTable – Registration Details
|
|-- JButton – Export

6.7 Application Event Flow



6.8 Layered Application Architecture

The application should use separate layers to improve code organization.

Presentation Layer

This layer contains the Swing GUI components. It accepts user input and displays application results.

Service Layer

This layer contains business rules such as validation, duplicate checking and registration processing.

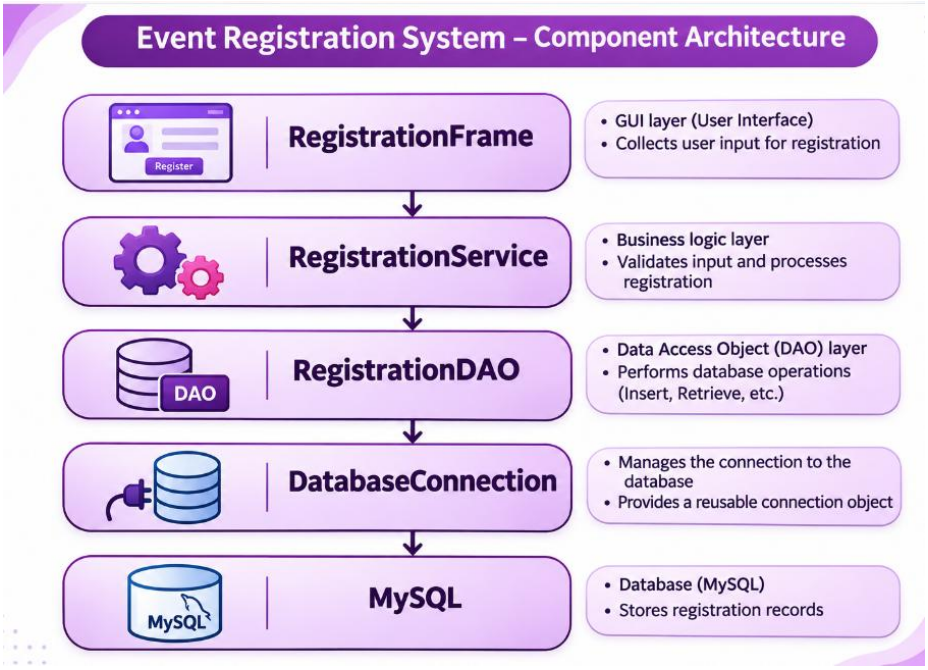
Data Access Layer

This layer handles database communication using JDBC and executes SQL queries.

Database Layer

MySQL stores student registration information permanently.

Proposed Class Structure



6.9 Class Responsibilities

Class	Main Responsibility
RegistrationFrame	Creates and manages the GUI.
RegistrationService	Performs validation and business operations.
RegistrationDAO	Handles SQL queries and database operations.
Registration	Represents registration details.
DatabaseConnection	Establishes a JDBC connection.
ExportService	Handles exporting registration information.

6.10 Input Validation

The application must validate student details before storing them in the database.

Validation rules include:

- Student ID must not be empty.
- Student name must be entered.
- Email address must follow a valid format.
- Department must be selected.
- Event must be selected.
- The student must not already be registered for the same event.

6.11 Duplicate Registration Prevention

Duplicate registration prevention is an important requirement.

Before inserting a new registration, the application should check whether the same student ID already exists for the selected event.

A database-level unique constraint should also be implemented on the combination of student ID and event name. This provides additional protection when multiple registration requests occur simultaneously.

6.12 Database Design

The application uses MySQL as its database management system.

Database Name: campus_event_db

Table Name: registrations

Column	Data Type	Description
registration_id	INT	Primary key
student_id	VARCHAR(30)	Student identification
student_name	VARCHAR(100)	Student name
email	VARCHAR(100)	Email address
department	VARCHAR(50)	Student department
event_name	VARCHAR(100)	Selected event
registration_date	TIMESTAMP	Registration date and time

Important database constraint: The combination of student ID and event name must be unique to prevent duplicate registrations.

6.13 Register Operation Pseudocode

```
BEGIN

  READ student ID
  READ student name
  READ email
  READ department
  READ event name

  IF student ID is empty THEN
    DISPLAY student ID error
    STOP
  END IF

  IF student name is empty THEN
    DISPLAY student name error
    STOP
  END IF

  IF email is invalid THEN
    DISPLAY email validation error
    STOP
  END IF

  IF event name is not selected THEN
    DISPLAY event selection error
    STOP
  END IF

  START background database operation

    CHECK whether the student is already registered
```

```
    IF duplicate record exists THEN
      DISPLAY duplicate registration message
    ELSE
      INSERT student registration into database
      DISPLAY successful registration message
    END IF

  END background operation

END
```

6.14 Filter Operation Pseudocode

```
BEGIN

  READ selected event name

  START background database operation

    RETRIEVE registrations for selected event

    CLEAR existing table records

    FOR each retrieved registration

      ADD registration details to JTable

    END FOR

  END background operation

END
```

6.15 Importance of SwingWorker

Database operations may take time depending on the number of records and database response.

If these operations are performed directly on the Event Dispatch Thread, the application may become unresponsive.

SwingWorker can execute long-running operations in the background while allowing the GUI to remain active.

This improves the overall user experience.

6.16 Testing and Expected Results

Test Case	Description	Expected Result
Test 1	Enter valid registration details.	Registration should be successful.
Test 2	Leave student ID empty.	Student ID error should appear.
Test 3	Enter an invalid email.	Email validation error should appear.
Test 4	Register the same student for the same event again.	Duplicate registration should be rejected.
Test 5	Filter by event name.	Matching registration records should appear.
Test 6	Export registration records.	Registration information should be exported.
Test 7	Retrieve a large number of records.	GUI should remain responsive.

6.17 Alternative Approach: Loading All Records vs Pagination

The application may retrieve all registration records at once or retrieve them page by page.

Approach 1: Loading all records

In this method, the application retrieves all 100,000 registration records and displays them in the JTable.

Advantages:

- All records are available in one table.
- Searching within already loaded data can be convenient.

Disadvantages:

- High memory consumption.
- Initial loading may take more time.
- Large table models may reduce interface responsiveness.

Time depends on database retrieval, data transfer and table rendering.

Memory complexity: O(N), where N is the total number of records.

Approach 2: Pagination using LIMIT and OFFSET

Pagination retrieves a limited number of records at a time.

For example, the application can retrieve 100 records per page. When the user selects Next, it retrieves the next group.

Advantages:

- Lower memory usage for displayed records.
- Faster initial display in many cases.
- Suitable for large registration databases.
- Works effectively with background processing.

Memory complexity for the displayed page: O(P), where P is the page size.

The number of pages is:

Number of Pages = Ceiling (Total Records / Page Size)

For 100,000 records with 100 records per page:

Number of Pages = 100,000 / 100 = 1,000 pages

6.18 Comparison

Parameter	**Load All Records**	**Pagination**
Data retrieval	Entire database result	Limited records
Memory usage	O(N)	O(P) for displayed records
Initial loading	May take longer	Usually faster
Responsiveness	May be affected	Better with SwingWorker
Scalability	Less suitable for large datasets	More suitable
Navigation	Large table	Next and Previous pages

PROGRAM:

```
import java.util.ArrayList;
import java.util.Scanner;
class Registration {
   String studentId;
   String name;
   String email;
   String department;
   String event;
   Registration(String studentId, String name,
            String email, String department,
```

```
            String event) {
        this.studentId = studentId;
        this.name = name;
        this.email = email;
        this.department = department;
        this.event = event;
    }
    void display() {
        System.out.println(
            studentId + " | " + name + " | " +
            email + " | " + department + " | " + event
        );
    }
}
public class Main {
    static ArrayList<Registration> records =
        new ArrayList<>();
    static void registerStudent(Scanner sc) {
        System.out.print("Enter Student ID: ");
        String id = sc.nextLine();
        System.out.print("Enter Student Name: ");
        String name = sc.nextLine();
        System.out.print("Enter Email: ");
        String email = sc.nextLine();
        System.out.print("Enter Department: ");
        String department = sc.nextLine();
        System.out.print("Enter Event Name: ");
        String event = sc.nextLine();
        if (id.isEmpty() || name.isEmpty() ||
            email.isEmpty() || department.isEmpty() ||
            event.isEmpty()) {
            System.out.println("Please fill all fields");
            return;
        }
        if (!email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$")) {
            System.out.println("Invalid email address");
            return;
        }
        for (Registration r : records) {
            if (r.studentId.equalsIgnoreCase(id) &&
                r.event.equalsIgnoreCase(event)) {
                System.out.println(
                    "Student already registered for this event"
                );
```

```
            return;
        }


    Registration r = new Registration(
        id, name, email, department, event
    );
    records.add(r);
    System.out.println("Registration Successful!");
}
static void viewRecords() {
    System.out.println("\n===== REGISTRATION RECORDS =====");
    if (records.isEmpty()) {
        System.out.println("No registrations available");
        return;
    }
    System.out.println(
        "Student ID | Name | Email | Department | Event"
    );
    for (Registration r : records) {
        r.display();
    }
}
static void filterRecords(Scanner sc) {
    System.out.print("Enter Event Name to Filter: ");
    String event = sc.nextLine();
    boolean found = false;
    System.out.println("\n===== FILTERED RECORDS =====");
    for (Registration r : records) {
        if (r.event.equalsIgnoreCase(event)) {
            r.display();
            found = true;
        }
    }
    if (!found) {
        System.out.println("No matching registrations found");
    }
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int choice;
    do {
        System.out.println("\n===== CAMPUS EVENT REGISTRATION =====");
        System.out.println("1. Register Student");
```

```
            System.out.println("2. View Registrations");
            System.out.println("3. Filter by Event");
            System.out.println("4. Exit");
           System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            sc.nextLine();
            switch (choice) {
                case 1:
                    registerStudent(sc);
                    break;
                case 2:
                    viewRecords();
                    break;
                case 3:
                    filterRecords(sc);
                    break;
                case 4:
                    System.out.println("Thank You!");
                    break;
                default:
                    System.out.println("Invalid Choice");
            }
        } while (choice != 4);
        sc.close();
    }
}
```

Main.java

Clear | Run ▶

```
                case 2:
                    viewRecords();
                    break;

                case 3:
                    filterRecords(sc);
                    break;

                case 4:
                    System.out.println("Thank You!");
                    break;

                default:
                    System.out.println("Invalid Choice");
            }

        } while (choice != 4);

        sc.close();
    }
}
```

Output

```
===== CAMPUS EVENT REGISTRATION =====
1. Register Student
2. View Registrations
3. Filter by Event
4. Exit
Enter Choice: 1
Enter Student ID: 101
Enter Student Name: OAVI
Enter Email: Oavi@gmail.com
Enter Department: cse(ai)
Enter Event Name: code zz
Registration Successful!

===== CAMPUS EVENT REGISTRATION =====
1. Register Student
2. View Registrations
3. Filter by Event
4. Exit
Enter Choice:

=== Session Ended Due To Inactivity ===
```

Conclusion: Pagination with SwingWorker is a suitable approach for the campus event registration application because it reduces the number of records loaded at once and improves the responsiveness of the graphical interface.

CONCLUSION

This assignment provides an understanding of event-driven GUI development using Java AWT and Swing. It explains how graphical components communicate with event listeners and how different event-handling approaches can be used to perform application operations.

The first task demonstrates the importance of separating graphical interface code from business logic. The second task explains how an appointment system can be designed using event sources, listeners and suitable GUI components. The third task focuses on selecting appropriate layout managers to create a responsive library search interface.

The fourth task explains how mouse events can be handled correctly and how drawing information can be preserved during repainting. The fifth task compares different event-handling techniques, including ActionListener, inner classes, anonymous inner classes and lambda expressions.

The final task demonstrates the design of a campus event registration application using Swing, JDBC and MySQL. It highlights the importance of input validation, duplicate prevention, layered architecture, database connectivity and background processing.

Overall, the assignment improves the ability to analyse GUI requirements, select appropriate components, design event handlers and develop maintainable Java desktop applications.